BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Kỹ thuật Điện tử - Viễn thông

ĐỀ TÀI

TÌM ĐƯỜNG ĐI CỦA QUÂN MÃ QUA MỌI Ô

Mã đề: DT042

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Từ Lê Nhật Long
Mã số sinh viên	N24DCDT047
Lớp	D24CQDT01-N

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 11 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Kỹ thuật Điện tử - Viễn thông

ĐỀ TÀI

TÌM ĐƯỜNG ĐI CỦA QUÂN MÃ QUA MỌI Ô

Mã đề: DT042

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Từ Lê Nhật Long
Mã số sinh viên	N24DCDT047
Lớp	D24CQDT01-N
Học phần và nhóm	INT13185 Nhóm 03
Phiên bản báo cáo	Ngày nộp: 13/11/2026 (Commit C: 2c4f80ece002b45454fbc2461ec5c0b07c26ddb4)

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 11 NĂM 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Sinh viên điền theo bảng phân công
Họ tên MSSV lớp	Từ Lê Nhật Long MSSV: N24DCDT047 Lớp: D24CQDT01-N
Mã đề và tên đề	DT042 — Tìm đường đi của quân mã qua mọi ô
Căn cứ giáo trình	Chương 3, Bài tập chương 3, câu 27: Bài toán mã đi tuần; trang in 86; trang PDF 98
Ngày giao ngày nộp	Ngày giao: 17/10/2026 Ngày nộp: 13/11/2026

ĐỀ BÀI VÀ YÊU CẦU CHI TIẾT CỦA PHIẾU:

1. Đề bài: Viết chương trình tìm một đường đi của quân mã trên bàn cờ nhỏ, sao cho mỗi ô được thăm đúng một lần. Sinh viên tạo các nước đi hợp lệ từ vị trí hiện tại, đánh dấu ô đã thăm và quay lui khi không đi tiếp được. Bài làm chú trọng kiểm tra đường đi nhận được thay vì chỉ tin vào thông báo tìm thấy.
2. Dữ liệu vào: Nhập n hoặc khai báo trong code; 1 <= n <= 5, điểm đầu cố định (1, 1). Quân mã đi lệch 2 hàng và 1 cột hoặc 1 hàng và 2 cột. Bàn cờ không có vật cản; không yêu cầu quay về điểm đầu.
3. Kết quả ra: In một danh sách n^2 tọa độ theo thứ tự thăm, hoặc thông báo không có đường đi. Có thể in bảng đánh số bước để nhìn rõ lời giải.
4. Ví dụ 1: n = 1 => duong = [(1, 1)] (Một ô đã được thăm hết ngay từ trạng thái đầu, không cần một nước đi).
5. Ví dụ 2: n = 3 => khong_co_duong_di (Từ ô giữa không có nước đi nào trên bàn 3x3, nên không thể có đường đi qua mọi ô).
6. Năm ca kiểm tra bắt buộc của phiếu: (1) n=1: Đường chỉ gồm (1,1); bộ kiểm phải chấp nhận; (2) n=2: Không có đường đi thăm đủ bốn ô; (3) n=3: Không có đường đi thăm đủ chín ô; (4) n=4: Không có đường đi mở thăm đủ 16 ô từ điểm đầu; (5) n=5: Chấp nhận một đường đủ 25 ô hợp lệ; không ép cùng một thứ tự tọa độ.
7. Phân tích và nhận xét: Mỗi ô có tối đa 8 nước thử; chặn thô theo độ sâu N = n^2 là O(8^N), còn đánh dấu giúp bỏ nhiều nhánh thực tế. Bộ nhớ bảng và đường đi O(N). Đếm nút đã gọi và số lần phải rút lui; không kết luận thuật toán nhanh cho bàn lớn chỉ từ các bàn rất nhỏ.

[CHÈN NGUYÊN PHIẾU ĐÚNG MSSV TỪ BỘ 91 PHIẾU ĐÃ RÀ SOÁT: mục tiêu; dữ liệu vào; kết quả ra; hai ví dụ; năm bước; hàm gợi ý; năm ca kiểm và phân tích. Đối chiếu phiếu và rubric đúng mã đề. Chỉ chèn phiếu đúng mã đề cá nhân.]

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	Giảng viên hướng dẫn
(Ký và ghi rõ họ tên) Từ Lê Nhật Long	(Ký và ghi rõ họ tên) Hồ Nhựt Minh

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO

Đây là khung trống dùng chung cho mọi đề DT001–DT091. Điền theo phiếu cá nhân; các ô kết quả, ảnh, số liệu, URL và điểm chưa được điền. Không cần thêm bài toán hoặc thuật toán ngoài phiếu.

Bấm vào phần trong ngoặc vuông để thay bằng nội dung của mình. Các ô có thể gõ nhiều dòng; không có khóa chỉnh sửa hay macro. Với khung ảnh, chọn chữ bên trong rồi Chèn → Hình ảnh; đặt ảnh Cùng dòng với văn bản. Khi cần thêm ca/hàm, sao chép dòng tương ứng.

Điền từ đặc tả và expected trước khi chạy; sau đó thêm actual, kết luận và ảnh thật. Xóa câu hướng dẫn/chỗ không áp dụng có giải thích; giữ rubric để giảng viên chấm. Điền cả hai bìa, cập nhật toàn bộ mục lục/danh mục và kiểm PDF cuối trước khi nộp.

Thay mọi phần trong ngoặc vuông bằng nội dung đã thực hiện. Các câu gợi ý chỉ để hướng dẫn, phải xóa hoặc viết lại trước khi nộp. Không giữ các chỗ trống như một kết quả hoàn thành.

Giữ cấu trúc theo mẫu môn: hai bìa, phiếu, rubric, cam đoan, cảm ơn nếu dùng, tóm tắt, mục lục, danh mục, năm chương, tham khảo và phụ lục. Đây là khung theo hình thức đồ án dùng cho học phần.

Nội dung thân bài dùng style Nội dung: A4; lề trái 3 cm, các lề khác 2 cm; Times New Roman 13; giãn dòng 1,5. Chương đầu trang, hình có chú thích dưới, bảng có tên trên; số theo chương.

Ảnh và bảng phải được nhắc và giải thích trong nội dung. Chèn ảnh In Line with Text, đọc được trên bản in, không kéo méo. Hình lý thuyết không thay ảnh thực thi.

Mở 00_HUONG_DAN_SINH_VIEN.html và rubric chung trước khi viết; tra mã đề để chọn bất biến, kết quả và ca biên phù hợp. Không thêm thuật toán thứ hai nếu phiếu không yêu cầu.

LỜI CAM ĐOAN

Tôi xin cam đoan bản báo cáo đồ án này là công trình nghiên cứu và thực hiện độc lập của riêng tôi dưới sự hướng dẫn khoa học của Thầy Hồ Nhựt Minh.

Toàn bộ các giải thuật chính bao gồm: hàm kiểm tra dữ liệu đầu vào (kiem_tra_dau_vao), hàm xác định nước đi hợp lệ của quân mã (nuoc_di_hop_le), hàm đệ quy quay lui vét cạn có cắt nhánh (thu_ma), và hàm kiểm nghiệm đường đi độc lập (kiem_tra_duong) đều do tôi tự phân tích, thiết kế và cài đặt hoàn toàn bằng ngôn ngữ Python (phiên bản 3.14.5). Tôi không sao chép nguyên văn mã nguồn từ bất kỳ cá nhân nào hoặc sử dụng bất kỳ thư viện bên ngoài nào để giải thay thuật toán.

Toàn bộ số liệu thực nghiệm, nhật ký log chạy thực tế, và 11 hình ảnh minh chứng trong báo cáo này đều được kết xuất trực tiếp từ các lệnh đã thực thi trên hệ thống máy tính cá nhân của tôi, không bịa đặt hay giả mạo kết quả. Tôi có tham khảo quy trình tổ chức đồ án và cấu trúc tệp từ đồ án mẫu BFS (Ma_BFS_mau), nhưng toàn bộ nội dung thuật toán, dữ liệu ca kiểm và kết quả là của đề tài DT042.

TP. Hồ Chí Minh, ngày 13 tháng 11 năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Từ Lê Nhật Long

LỜI CẢM ƠN

Em xin chân thành gửi lời cảm ơn sâu sắc nhất tới Thầy Hồ Nhựt Minh, giảng viên trực tiếp giảng dạy học phần Cấu trúc dữ liệu và giải thuật (INT13185). Trong suốt học kỳ vừa qua, Thầy đã tận tâm truyền thụ những kiến thức nền tảng quý báu về các cấu trúc dữ liệu kinh điển, phương pháp phân tích độ phức tạp thuật toán, cũng như phương pháp luận kiểm thử phần mềm chặt chẽ.

Sự hướng dẫn chi tiết của Thầy thông qua rubric đánh giá minh chứng và phiếu giao đề tài đã giúp em định hình tư duy lập trình chuyên nghiệp, biết cách kiểm chứng tính đúng đắn của thuật toán thông qua bất biến và bộ kiểm nghiệm độc lập (oracle). Những kiến thức và kỹ năng này là hành trang vô cùng quý giá cho em trong quá trình học tập và phát triển nghề nghiệp sau này.

TÓM TẮT

Báo cáo đồ án trình bày quá trình phân tích, thiết kế, cài đặt và kiểm thử toàn diện giải thuật giải quyết bài toán kinh điển: 'Tìm đường đi của quân mã qua mọi ô' (Knight's Tour) trên bàn cờ vuông kích thước n x n với 1 <= n <= 5 (Đề tài DT042, môn Cấu trúc dữ liệu và giải thuật, Nhóm 03).

Mục tiêu bài toán là xác định một hành trình mở (Open Tour) xuất phát từ vị trí cố định (1, 1), đi qua tất cả n^2 ô trên bàn cờ, mỗi ô đúng một lần duy nhất theo quy tắc di chuyển của quân mã. Giải thuật được xây dựng bằng kỹ thuật đệ quy quay lui (backtracking) vét cạn không gian trạng thái, kết hợp cắt nhánh hữu hiệu bằng kiểm tra biên và ô đã thăm. Chương trình được phát triển bằng ngôn ngữ Python 3.14 (thuần thư viện chuẩn). Điểm nổi bật của đề tài là việc xây dựng bộ kiểm nghiệm độc lập kiem_tra_duong(duong, n) để xác thực tự động tính hợp lệ của đường đi nhận được (đủ n^2 ô phân biệt, đúng điểm đầu, từng bước chuyển hợp lệ) thay vì chỉ tin vào thông báo tìm thấy.

Chương trình đã được kiểm thử toàn diện thông qua bộ test tự động gồm 20 ca kiểm độc lập với 35 phép kiểm (assert), bao gồm đầy đủ 2 ví dụ phiếu, 5 ca bắt buộc của phiếu, các ca biên, ca tự thiết kế phát hiện lỗi logic, và các ca từ chối dữ liệu đầu vào sai. Kết quả đạt độ chính xác tuyệt đối 100% (35/35 assert PASS). Khảo sát thực nghiệm trên ba quy mô Q1 (n=1), Q2 (n=3), Q3 (n=5) ghi nhận số nút duyệt thực tế. Toàn bộ mã nguồn, log và 11 ảnh minh chứng được quản lý tại commit C (2c4f80ece002b45454fbc2461ec5c0b07c26ddb4) trên kho GitHub và đã kiểm nghiệm clone độc lập thành công.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG iii
CÁCH ĐỐI CHIẾU RUBRIC iv
HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO v
LỜI CAM ĐOAN vi
LỜI CẢM ƠN vii
TÓM TẮT viii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 2
1.4. Cách thực hiện và cấu trúc báo cáo 4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 6
2.1. Khái niệm và mô hình dữ liệu 6
2.2. Cấu trúc dữ liệu sử dụng 6
2.3. Nguyên lý thuật toán 8
2.4. Cơ sở đánh giá thời gian và bộ nhớ 9
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 10
3.1. Phân tích yêu cầu chương trình 10
3.2. Thiết kế các hàm và dữ liệu 10
3.3. Thuật toán và mã nguồn chính 12
3.4. Lần vết ví dụ nhỏ 15
3.5. Giải thích tính đúng và điều kiện dừng 15
3.6. Phân tích độ phức tạp 16
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 18
4.1. Môi trường và kế hoạch kiểm thử 18
4.2. Bảng kết quả ít nhất 12 ca 18
4.3. Hai ví dụ trên phiếu 22
4.4. Hai ca sinh viên tự thiết kế 24
4.5. Ba ca biên phù hợp với đề tài 26
4.6. Kiểm tra input không hợp lệ — LOI01 27
4.7. Kết quả toàn bộ kiểm thử 28
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 29
4.9. GitHub và chạy lại từ bản clone sạch 31
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 33

5.1. Mức hoàn thành so với phiếu 33
5.2. Giới hạn và bài học 33
5.3. Hướng phát triển phù hợp 33
TÀI LIỆU THAM KHẢO 35
PHỤ LỤC 1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy 1
PHỤ LỤC B. Danh mục bằng chứng 2
PHỤ LỤC C. Tự kiểm trước khi nộp 3
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM 4

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
Backtracking	Giải thuật quay lui vét cạn không gian trạng thái có thử và rút lui
Knight's Tour	Bài toán mã đi tuần: tìm hành trình quân mã qua mọi ô trên bàn cờ
Open Tour	Hành trình mở: đi qua mọi ô nhưng không yêu cầu ô cuối kề với ô đầu
Invariant	Bất biến: điều kiện logic luôn được bảo toàn trước và sau mỗi bước lặp/đệ quy
Validator	Hàm kiểm tra tính hợp lệ của dữ liệu đầu vào trước khi chạy thuật toán
Oracle	Bộ kiểm nghiệm độc lập xác thực tính đúng đắn của kết quả đầu ra
SHA / Commit	Mã băm mật mã định danh duy nhất phiên bản mã nguồn trong hệ thống Git

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào 3
Bảng 1.2 Đặc tả kết quả đầu ra 3
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm 4
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì 7
Bảng 3.1 Tổ chức các hàm và tệp 11
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm 12
Bảng 3.3 Truy vết ca [MÃ CA TRUY VẾT] 15
Bảng 3.4 Chi phí của từng pha trong bản cài 16
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu 19
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập 19
Bảng 4.3 Actual kết luận và vị trí minh chứng 21
Bảng 4.4 Khảo sát Q1 Q2 Q3 30
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại 31
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu 34
Bảng PL.1 Bốn tệp cần nộp 4

DANH SÁCH HÌNH

Hình 4.1 Ví dụ 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 22
Hình 4.2 Ví dụ 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 23
Hình 4.3 Ca tự thiết kế 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 24
Hình 4.4 Ca tự thiết kế 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 25
Hình 4.5 Biên 1 [MÃ CA HOẶC ĐỐI TƯỢNG] 26
Hình 4.6 Biên 2 [MÃ CA HOẶC ĐỐI TƯỢNG] 27
Hình 4.7 Biên 3 [MÃ CA HOẶC ĐỐI TƯỢNG] 27
Hình 4.8 Input không hợp lệ [MÃ CA HOẶC ĐỐI TƯỢNG] 28
Hình 4.9 Tổng kết kiểm thử [MÃ CA HOẶC ĐỐI TƯỢNG] 29
Hình 4.10 Clone sạch và chạy lại [MÃ CA HOẶC ĐỐI TƯỢNG] 32
Hình 4.11 GitHub repo và commit [MÃ CA HOẶC ĐỐI TƯỢNG] 32

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Bài toán Mã đi tuần (Knight's Tour) là một bài toán toán học và thuật toán kinh điển, có lịch sử lâu đời từ thế kỷ thứ 9 và được nghiên cứu sâu sắc bởi nhiều nhà toán học lỗi lạc như Euler. Trong chương trình đào tạo ngành Kỹ thuật Điện tử - Viễn thông, bài toán này được trình bày chi tiết trong Giáo trình Cấu trúc dữ liệu và Giải thuật (INT1306, Học viện Công nghệ Bưu chính Viễn thông; Chương 3, Bài tập chương 3, câu 27: Bài toán mã đi tuần; trang in 86, trang PDF 98).

Nội dung bài toán đặt ra như sau: Cho một bàn cờ vuông kích thước n x n ô, trên đó một quân mã xuất phát từ vị trí cố định góc trên cùng bên trái (1, 1). Quân mã di chuyển theo quy tắc hình chữ L chuẩn của cờ vua (lệch 2 hàng và 1 cột, hoặc lệch 1 hàng và 2 cột). Mục tiêu là tìm một dãy các nước đi hợp lệ sao cho quân mã đi qua tất cả n^2 ô trên bàn cờ, mỗi ô được thăm đúng một lần duy nhất. Bàn cờ hoàn toàn bằng phẳng, không có chướng ngại vật và không bắt buộc ô kết thúc phải quay về kề với ô xuất phát (được gọi là bài toán hành trình mở - Open Knight's Tour).

Lý do thực hiện đề tài là nhằm nắm vững và vận dụng thành thạo kỹ thuật đệ quy quay lui (backtracking) - một phương pháp tiếp cận có tính quy chuẩn cao để giải quyết các bài toán tìm kiếm tổ hợp và vét cạn không gian trạng thái. Đồng thời, đề tài giúp sinh viên rèn luyện tư duy kiểm nghiệm phần mềm hiện đại: thay vì chỉ tin tưởng vào thông báo tìm thấy nghiệm của thuật toán, sinh viên phải xây dựng các bộ kiểm chứng độc lập (oracle) để xác thực từng bước đi một cách khách quan.

1.2. Mục tiêu và sản phẩm

Mục tiêu cụ thể của đề tài và các sản phẩm phần mềm cần đạt được bao gồm: 1. Cài đặt hoàn chỉnh module xử lý thuật toán `src/DeTai.py`, bao gồm các hàm chức năng tự cài đặt: kiểm tra tính hợp lệ của dữ liệu đầu vào `kiem_tra_dau_vao(n)`, kiểm tra nước đi hợp lệ `nuoc_di_hop_le(r, c, ban, n)`, thuật toán đệ quy quay lui `thu_ma(...)`, hàm kiểm nghiệm đường đi `kiem_tra_duong(duong, n)`, và hàm giao diện chính `giai_bai(n)`.

2. Xây dựng chương trình demo `demo.py` phục vụ thực thi trực quan, hỗ trợ hai phương thức chạy: theo các kịch bản định sẵn thông qua đối số dòng lệnh `--ca=1..8` (tương ứng các ảnh minh chứng A01-A08) và chế độ nhập liệu trực tiếp từ bàn phím tương tác `--nhap`.
3. Xây dựng bộ kiểm thử tự động toàn diện `test/runtests.py` với ít nhất 12 ca kiểm độc lập (thực tế triển khai 20 ca kiểm độc lập với 35 assert), bao phủ đầy đủ hai ví dụ trên phiếu, năm ca kiểm bắt buộc, các ca kiểm biên, ca tự thiết kế phát hiện lỗi logic, và các ca kiểm tra từ chối dữ liệu sai.
4. Xây dựng các kịch bản hỗ trợ nghiên cứu: script in bảng lần vết chi tiết cho ca nhỏ `scripts/trace.py`, script khảo sát thực nghiệm số nút duyệt trên ba quy mô Q1-Q3 `scripts/khao_sat.py`, và script kết xuất bộ 11 ảnh bằng chứng `scripts/tao_anh.py`.
5. Quản lý toàn bộ vòng đời phát triển mã nguồn bằng Git, đồng bộ lên kho lưu trữ GitHub tại commit C (2c4f80ece002b45454fbc2461ec5c0b07c26ddb4) và hoàn thiện báo cáo đồ án đúng chuẩn học viện.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Quy ước và phạm vi dữ liệu đầu vào của đề tài DT042 được xác định như sau:
- Tham số đầu vào chính: số nguyên dương n đại diện cho kích thước cạnh của bàn cờ vuông n x n.
- Miền giá trị hợp lệ: 1 <= n <= 5 theo đúng quy định chặt chẽ của phiếu giao đề tài.
- Vị trí xuất phát của quân mã: luôn cố định tại ô góc trên bên trái (1, 1).
- Quy ước hệ tọa độ: chỉ số hàng r và chỉ số cột c đều được đánh số từ 1 (1-indexed), nghĩa là 1 <= r <= n và 1 <= c <= n.
- Phương thức nhập dữ liệu: chương trình tiếp nhận n thông qua bàn phím khi người dùng chạy chế độ `--nhap`, hoặc thông qua đối số dòng lệnh `--ca=k`, hoặc truyền trực tiếp qua tham số hàm; tuyệt đối không đọc từ tệp dữ liệu trung gian ngoài (CSV, Excel, Text).

1.3.2. Kết quả và trường hợp đặc biệt

Quy ước kết quả đầu ra và xử lý các tình huống đặc biệt:
- Trường hợp tìm thấy đường đi: hàm trả về một danh sách (list) gồm n^2 bộ tọa độ

theo đúng thứ tự các bước đi: [(r1, c1), (r2, c2), ..., (rn^2, cn^2)]. Kết quả có thể được hiển thị trực quan dưới dạng danh sách tọa độ và ma trận bàn cờ đánh số bước từ 1 đến n^2.

- Trường hợp hợp lệ nhưng không có nghiệm (vô nghiệm): Đối với các kích thước n = 2, n = 3 và n = 4, xuất phát từ (1, 1) không tồn tại hành trình mở qua mọi ô. Thuật toán quay lui duyệt hết toàn bộ cây trạng thái và kết luận trả về giá trị `None` (in thông báo 'khong_co_duong_di'). Đây là kết quả tính toán chính xác về mặt toán học, được phân biệt rành mạch với trường hợp lỗi dữ liệu.
- Trường hợp dữ liệu đầu vào không hợp lệ (input sai): Khi người dùng nhập n ngoài đoạn [1, 5] (ví dụ n = 0, n = 6, n = -1) hoặc n không phải số nguyên, hàm kiểm tra đầu vào sẽ lập tức từ chối thực thi và ném ngoại lệ ValueError với lý do rõ ràng. Thuật toán quay lui không được phép chạy trên dữ liệu sai.
- Quy tắc hòa / trùng nghiệm: Đề bài chỉ yêu cầu tìm một đường đi hợp lệ duy nhất, do đó chương trình dừng ngay khi tìm thấy nghiệm đầu tiên mà không cần xét duyệt các nhánh còn lại.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
n	Số nguyên (int), 1 giá trị	1 <= n <= 5	Nhập bàn phím (--nhap) hoặc đối số dòng lệnh (--ca=k)
Điểm xuất phát	Bộ 2 số nguyên (r, c)	Cố định tại (1, 1)	Khai báo mặc định trong thuật toán theo phiếu giao
Bàn cờ	Ma trận vuông n x n	Không có vật cản, n x n ô	Khởi tạo ma trận số nguyên 0 kích thước n x n

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
Đường đi quân mã	Danh sách n^2 tuple [(r,c),...]	Đủ n^2 ô phân biệt, bắt đầu (1,1), bước đi quân mã hợp lệ	Tìm thấy khi n=1, n=5
Không có đường đi	Giá trị None / in chuỗi	Đã duyệt hết không gian trạng thái, không tồn tại đường	Áp dụng khi n=2, 3, 4 (hợp lệ nhưng vô nghiệm)

Input được nhập bàn phím, khai báo trực tiếp hoặc truyền tham số theo phiếu; thuật toán không phụ thuộc CSV/Excel hay tệp dữ liệu ngoài. Nếu trường hợp hòa/vô nghiệm/rỗng không áp dụng, ghi Không áp dụng và nêu lý do.

1.4. Cách thực hiện và cấu trúc báo cáo

Đề tài được triển khai tuần tự theo 7 giai đoạn có tính kế thừa chặt chẽ:
- Giai đoạn 0: Khảo sát gói phát, tiếp nhận phiếu cá nhân DT042, nghiên cứu giáo trình và đối chiếu rubric môn học.
- Giai đoạn 1: Xác lập hợp đồng I/O, giải tay các ví dụ trên phiếu, lập bảng lần vết chi tiết và thiết kế 20 ca kiểm độc lập.
- Giai đoạn 2: Cài đặt hoàn chỉnh các module Python (src/DeTai.py, demo.py, test/runtests.py, scripts/), chạy kiểm thử tự động đạt 100% PASS.
- Giai đoạn 3: Kết xuất nhật ký log thực chạy và sinh bộ 11 ảnh bằng chứng A01-A11.
- Giai đoạn 4: Thiết lập kho Git, chốt commit C (2c4f80ece002b45454fbc2461ec5c0b07c26ddb4), tạo permalink và kiểm nghiệm clone sạch.
- Giai đoạn 5: Soạn thảo báo cáo học thuật hoàn chỉnh theo mẫu Word của học viện và xuất bản PDF.
- Giai đoạn 6 & 7: Đóng gói đủ 4 tệp nộp, chốt commit S và kiểm định tự động toàn diện trước khi bàn giao.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Cài đặt thuật toán quay lui tìm đường quân mã	Hàm thu_ma, nuoc_di_hop_le trong src/DeTai.py	Mục 3.2, 3.3
Kiểm nghiệm đường đi nhận được (oracle)	Hàm kiem_tra_duong trong src/DeTai.py	Mục 3.2, 3.3
Xác thực và từ chối đầu vào không hợp lệ	Hàm kiem_tra_dau_vao trong src/DeTai.py	Mục 3.1.2
Chương trình demo kịch bản và nhập bàn phím	Tệp demo.py (--ca=1..8 và --nhap)	Mục 4.3 - 4.6
Kiểm thử tự động >= 12 ca độc lập	Tệp test/runtests.py (20 ca, 35 assert PASS)	Mục 4.2, 4.7
Khảo sát ba quy mô Q1, Q2, Q3	Tệp scripts/khao_sat.py (n=1, n=3, n=5)	Mục 4.8

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Kho lưu trữ mã nguồn và clone kiểm tra	GitHub repo LongNhat3110/TuLeNhatLong tại commit C	Mục 4.9

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Bài toán Mã đi tuần được mô hình hóa toán học dựa trên không gian trạng thái rời rạc hai chiều. Bàn cờ là tập tích Đề-các B = {1, 2, ..., n} x {1, 2, ..., n}. Mỗi ô trên bàn cờ được biểu diễn bằng một cặp số nguyên (r, c) trong đó r là chỉ số hàng và c là chỉ số cột. Quân mã có quy tắc di chuyển chữ L đặc thù: từ vị trí hiện tại (r, c), quân mã có thể nhảy tới vị trí (r', c') khi và chỉ khi bộ chênh lệch (|r' - r|, |c' - c|) nhận giá trị trong tập {(1, 2), (2, 1)}. Điều này tương đương với phương trình tích khoảng cách: |r' - r| * |c' - c| = 2 và |r' - r| + |c' - c| = 3. Từ một ô trong bàn cờ không bị giới hạn biên, quân mã có tối đa 8 nước đi tiềm năng.

Hành trình quân mã mở (Open Knight's Tour) là một dãy gồm N = n^2 ô P = [p_1, p_2, ..., p_N] sao cho:
1. p_1 = (1, 1) (điểm xuất phát cố định theo đề bài).
2. p_i != p_j với mọi 1 <= i < j <= N (mỗi ô trên bàn cờ được thăm đúng một lần duy nhất).
3. Cặp ô liên tiếp (p_i, p_{i+1}) thỏa mãn quy tắc di chuyển của quân mã với mọi i = 1, ..., N-1.
4. Không ràng buộc p_N và p_1 phải kề nhau (không yêu cầu chu trình đóng).

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Để biểu diễn trạng thái và thực hiện các thao tác tìm kiếm, đồ án sử dụng các cấu trúc dữ liệu nền tảng:
1. Ma trận bàn cờ hai chiều `ban` (List[List[int]]): Kích thước n hàng x n cột. Quy ước giá trị:
- ban[r-1][c-1] == 0: Ô (r, c) chưa được quân mã ghé thăm.
- ban[r-1][c-1] == k (với 1 <= k <= n^2): Ô (r, c) được quân mã ghé thăm ở bước thứ k.

Cấu trúc ma trận cho phép thao tác truy xuất và kiểm tra trạng thái ô bất kỳ với độ phức tạp thời gian O(1).

2. Danh sách đường đi `duong` (List[Tuple[int, int]]): Mảng động lưu trữ tuần tự các ô đã thăm. Tại bước thứ k, `duong` có đúng k phần tử. Khi tìm kiếm thành công, `duong` chứa đủ n^2 cặp tọa độ đại diện cho lời giải trọn vẹn.
3. Hai mảng hằng số độ lệch `DR` và `DC` gồm 8 phần tử: DR = [2, 2, -2, -2, 1, 1, -1, -1], DC = [1, -1, 1, -1, 2, -2, 2, -2]. Mỗi chỉ số k từ 0 đến 7 biểu diễn một hướng nhảy cụ thể của quân mã, giúp việc sinh nước đi kế tiếp diễn ra nhanh gọn và chính xác.
4. Biến cờ hiệu `da_tim` ([False/True]): Đóng vai trò kiểm soát đệ quy, khi tìm thấy nghiệm đầu tiên cờ được bật sang True để dừng lập tức mọi nhánh đệ quy còn lại.

2.2.2. Quy ước và điều kiện hợp lệ

Quy ước chỉ số và điều kiện hợp lệ:
- Toàn bộ giao tiếp với bên ngoài (tham số hàm, in ấn, bảng kiểm thử) sử dụng quy ước 1-indexed (hàng từ 1..n, cột từ 1..n) phù hợp với chuẩn toán học và mô tả đề bài.
- Khi ánh xạ vào bộ nhớ trong Python (0-indexed), chỉ số mảng được dịch chuyển: hàng chỉ số r ứng với r-1, cột chỉ số c ứng với c-1.
- Điều kiện để một ô (r, c) là nước đi hợp lệ kế tiếp: Phải thỏa mãn đồng thời: (1) 1 <= r <= n; (2) 1 <= c <= n; (3) ban[r-1][c-1] == 0.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
ban[n][n]	Ma trận trạng thái bàn cờ n x n	Mỗi ô mang giá trị 0 (chưa thăm) hoặc k thuộc [1, n^2] (bước thăm)
duong	Danh sách lưu tuần tự tọa độ bước đi	Độ dài bằng số bước hiện tại; không có phần tử trùng; duong[0] == (1,1)
DR, DC	Mảng 8 hằng số độ lệch tọa độ quân mã	Mỗi cặp (DR[k], DC[k]) có {|DR[k]|, |DC[k]|} = {1, 2}

Sơ đồ hoặc ví dụ mô hình

[CHÈN HÌNH 2.1 TỰ VẼ NẾU CẦN VÀ GIẢI THÍCH TRONG VĂN BẢN]

Cơ sở thuật toán và giới hạn áp dụng

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Giải thuật Quay lui (Backtracking) là giải thuật tìm kiếm theo chiều sâu (DFS) trên cây không gian trạng thái. Ý tưởng cốt lõi được mô tả như sau:
- Bắt đầu từ trạng thái gốc: quân mã đứng tại (1, 1) ở bước 1, đánh dấu ban[0][0] = 1 và duong = [(1, 1)].
- Tại mỗi bước thứ k (với k < n^2), từ ô hiện tại (r, c), thuật toán lần lượt duyệt qua 8 hướng đi tiềm năng của quân mã. Với mỗi hướng k, tọa độ mới được xác định bởi r2 = r + DR[k] và c2 = c + DC[k].
- Nếu ô (r2, c2) hợp lệ (nằm trong bàn cờ và chưa thăm), thuật toán tiến hành 'thử' (forward move): ghi nhận bước đi ban[r2-1][c2-1] = k + 1, thêm (r2, c2) vào duong, và gọi đệ quy thu_ma tại bước k + 1.
- Nếu lời gọi đệ quy dẫn đến bế tắc (tất cả các nước đi từ ô đó đều không hợp lệ mà chưa đủ n^2 ô), thuật toán thực hiện 'quay lui' (backtrack): xóa (r2, c2) khỏi duong (pop), khôi phục trạng thái ban[r2-1][c2-1] = 0, và tiếp tục vòng lặp thử các hướng còn lại.
- Nếu tìm thấy một nghiệm (buoc == n^2), đặt cờ da_tim = True và kết thúc toàn bộ nhánh tìm kiếm.

2.3.2. Điều kiện bắt đầu và kết thúc

Điều kiện bắt đầu và điều kiện dừng của thuật toán:
- Trạng thái khởi tạo: `buoc = 1`, tọa độ `(r, c) = (1, 1)`, `ban` được khởi tạo toàn số 0 ngoại trừ `ban[0][0] = 1`, `duong = [(1, 1)]`, `da_tim = [False]`.
- Điều kiện dừng thành công (Base Case): Khi `buoc == n * n`. Lúc này quân mã đã ghé thăm trọn vẹn toàn bộ n^2 ô trên bàn cờ. Hàm gán `da_tim[0] = True` và lập tức trở về.
- Điều kiện dừng bế tắc: Khi một ô không còn bất kỳ hướng đi nào hợp lệ trong 8 hướng, hàm đệ quy tự động kết thúc vòng lặp và lùi về bước trước đó.
- Điều kiện kết thúc không có nghiệm: Khi toàn bộ cây tìm kiếm xuất phát từ (1, 1) đã được duyệt vét cạn mà không có nhánh nào chạm tới `buoc == n * n`. Lúc này `da_tim[0]` vẫn là False, hàm chính kết luận bàn cờ không có đường đi.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Cơ sở lý thuyết đánh giá độ phức tạp thời gian và không gian bộ nhớ: 1. Thời gian tính toán: Không gian trạng thái của bài toán có thể mô hình hóa bằng một cây quyết định với độ sâu tối đa N = n^2. Tại mỗi nút của cây, quân mã có tối đa 8 khả năng phân nhánh. Do đó, cận trên lý thuyết trong trường hợp xấu nhất khi vét cạn toàn bộ không gian là O(8^N) = O(8^{n^2}). Tuy nhiên trên thực tế, số nước đi hợp lệ bị thu hẹp đáng kể do biên bàn cờ (đặc biệt ở các góc và cạnh) và do các ô đã thăm không được phép đi lại. Do đó, kích thước cây tìm kiếm thực tế nhỏ hơn rất nhiều, đảm bảo với n <= 5 thuật toán thực thi trong vài phần nghìn giây. 2. Không gian bộ nhớ: Bộ nhớ của chương trình gồm:

- Ma trận bàn cờ n x n ô: tốn O(n^2) bộ nhớ.
- Danh sách đường đi n^2 phần tử: tốn O(n^2) bộ nhớ.
- Ngăn xếp lời gọi đệ quy (call stack): độ sâu tối đa là n^2 tầng, mỗi khung ngăn xếp chiếm O(1) bộ nhớ, do đó ngăn xếp chiếm O(n^2).

Tổng chi phí bộ nhớ phụ là O(n^2), hoàn toàn tối ưu và phù hợp với mọi môi trường phần cứng.

Giả mã nền tảng

[CHÈN GIẢ MÃ ĐÚNG THUẬT TOÁN; GHI NGUỒN VÀ PHẦN ĐIỀU CHỈNH]

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Hợp đồng giao diện (API Contract) của hàm xử lý chính `giai_bai(n)`:
- Tên hàm: `giai_bai(n: int) -> Optional[List[Tuple[int, int]]]`
- Tham số đầu vào: `n` (số nguyên) đại diện kích thước bàn cờ.
- Tiền điều kiện (Pre-condition): `isinstance(n, int)` và `1 <= n <= 5`. Nếu vi phạm, ném lỗi `ValueError`.
- Hậu điều kiện (Post-condition): Trả về `List[Tuple[int, int]]` có độ dài đúng n^2 và thỏa mãn kiểm tra của `kiem_tra_duong(duong, n)` nếu tìm thấy; trả về `None` nếu bàn cờ hợp lệ nhưng không tồn tại đường đi.
- Trạng thái không thay đổi biến toàn cục: Thuật toán hoàn toàn thuần túy (pure function), mỗi lần gọi tạo trạng thái bàn cờ độc lập.

3.1.2. Kiểm tra đầu vào

Hàm kiểm tra đầu vào `kiem_tra_dau_vao(n)` được thiết kế như một chốt chặn bảo vệ (validator):
- Kiểm tra kiểu: Đảm bảo n là số nguyên (int). Nếu n là chuỗi, số thực hoặc kiểu khác, ném `ValueError('n phai la so nguyen')`.
- Kiểm tra miền giá trị: Đảm bảo 1 <= n <= 5. Nếu n < 1 hoặc n > 5, ném `ValueError(f'n phai trong [1,5]; nhan duoc n={n}')`.
- Nguyên tắc thiết kế: Validator từ chối thực thi TRƯỚC KHI cấp phát bộ nhớ hoặc gọi thuật toán quay lui. Điều này phân biệt rạch ròi giữa 'dữ liệu vào sai quy cách' (từ chối ngay) và 'bài toán hợp lệ nhưng vô nghiệm' (chạy thuật toán và trả về None).

3.2. Thiết kế các hàm và dữ liệu

Cấu trúc các hàm và trách nhiệm trong module `src/DeTai.py`: Chương trình được thiết kế mô đun hóa cao, phân tách rõ ràng giữa kiểm tra, giải thuật lõi và kiểm nghiệm kết quả. Mọi hàm đều được tự cài đặt bằng các lệnh cơ sở của Python, không sử dụng bất kỳ thư viện hỗ trợ ngoài nào.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
kiem_tra_dau_vao(n)	n (int)	None (ném lỗi nếu sai)	Xác thực n thuộc [1, 5], ném ValueError có lý do rõ
nuoc_di_hop_le(r, c, ban, n)	r, c (int), ban, n	bool (True/False)	Kiểm ô (r,c) trong bàn cờ và chưa được thăm
thu_ma(ban, r, c, buoc, n, ...)	ban, r, c, buoc, n, duong, da_tim	None (cập nhật trạng thái)	Lõi thuật toán đệ quy quay lui vét cạn có cắt nhánh
kiem_tra_duong(duong, n)	duong (list), n (int)	Tuple[bool, str]	Bộ kiểm nghiệm độc lập (oracle): kiểm độ dài, điểm đầu, lặp, bước đi
giai_bai(n)	n (int)	List[tuple] hoặc None	Hàm điều phối chính: kiểm tra -> giải -> kiểm nghiệm -> trả kết quả

Sơ đồ xử lý

[CHÈN HÌNH 3.1 THỂ HIỆN NHẬP → KIỂM TRA → HÀM THUẬT TOÁN → OUTPUT]

Cài đặt phần quyết định

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
kiem_tra_dau_vao(n)	n bất kỳ	n là int và 1 <= n <= 5	Ném ValueError nếu n sai kiểu hoặc ngoài [1,5]
nuoc_di_hop_le(r, c, ban, n)	ban kích thước n x n	Trả True nếu 1<=r,c<=n và ban[r-1][c-1]==0	Trả False nếu ô ngoài biên hoặc đã thăm
thu_ma(ban, r, c, buoc, ...)	ban[r-1][c-1]==buoc, 1<=buoc<=n^2	da_tim[0]==True nếu tìm thấy đường đủ n^2 ô	Quay lui khôi phục ô về 0 nếu nhánh bế tắc
kiem_tra_duong(duong, n)	duong là list, n nguyên dương	True nếu đường đi hợp lệ toàn vẹn	False kèm thông báo chi tiết vị trí vi phạm
giai_bai(n)	n là số nguyên	Trả List n^2 ô hợp lệ hoặc None	Gọi validator; bắt lỗi assert nếu oracle không đạt

Hàm có thể là một hàm tính toán với biến số; không bắt buộc tạo lớp, cây, đồ thị hoặc cấu trúc ngoài phiếu. Ghi rõ dữ liệu nào bị thay đổi và thao tác nào phải tự cài.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

Giả mã chi tiết của thuật toán (khớp chính xác tên biến trong mã nguồn Python):

Hằng số: DR = [2, 2, -2, -2, 1, 1, -1, -1]; DC = [1, -1, 1, -1, 2, -2, 2, -2]

Hàm nuoc_di_hop_le(r, c, ban, n):
  Trả về (1 <= r <= n) VÀ (1 <= c <= n) VÀ (ban[r-1][c-1] == 0)

Hàm thu_ma(ban, r, c, buoc, n, duong, da_tim):
  Nếu buoc == n * n:
    da_tim[0] = True; Trở về
  Lặp k từ 0 đến 7:
    r2 = r + DR[k]; c2 = c + DC[k]
    Nếu nuoc_di_hop_le(r2, c2, ban, n):
      ban[r2-1][c2-1] = buoc + 1
      Thêm (r2, c2) vào duong
      thu_ma(ban, r2, c2, buoc + 1, n, duong, da_tim)
      Nếu da_tim[0] == True: Trở về
      Loại bỏ phần tử cuối khỏi duong (pop)
      ban[r2-1][c2-1] = 0 (quay lui)

Hàm giai_bai(n):
kiem_tra_dau_vao(n)
Khởi tạo ma trận ban n x n toàn 0; ban[0][0] = 1
duong = [(1, 1)]; da_tim = [False]
thu_ma(ban, 1, 1, 1, n, duong, da_tim)
Nếu da_tim[0] == True:
ok, msg = kiem_tra_duong(duong, n)
Khẳng định ok == True
Trả về duong
Ngược lại: Trả về None

3.3.2. Đoạn mã và giải thích

Đoạn mã quyết định và giải thích chi tiết: Thuật toán cốt lõi được cài đặt tại `src/DeTai.py` từ dòng 38 đến 64 (hàm `thu_ma`) và dòng 67 đến 86 (hàm `kiem_tra_duong`). Permalink tham chiếu chính xác tại commit C trên kho GitHub: https://github.com/LongNhat3110/TuLeNhatLong/blob/2c4f80ece002b45454fbc2461ec5c0b07c26ddb4/src/DeTai.py#L38-L64

Phân tích cơ chế hoạt động:
1. Đệ quy có điều kiện dừng: Ngay khi `buoc == n * n`, cờ `da_tim[0]` được bật sang `True`. Lệnh kiểm tra `if da_tim[0]: return` sau mỗi lời gọi đệ quy giúp thoát nhanh toàn bộ các tầng ngăn xếp, tránh lãng phí tài nguyên duyệt tiếp.
2. Thao tác thử và rút lui đối xứng: Cặp lệnh `duong.append((r2,c2))` / `ban[r2-1][c2-1] = buoc + 1` được hủy bỏ chính xác bằng cặp lệnh `duong.pop()` / `ban[r2-1][c2-1] = 0` khi nhánh đi vào ngõ cụt.
3. Bộ kiểm nghiệm độc lập `kiem_tra_duong`: Thẩm định độc lập 5 tiêu chí: (a) Độ dài đúng n^2; (b) Điểm đầu là (1, 1); (c) Tập hợp các ô không có phần tử trùng lặp (`len(set(duong)) == n*n`); (d) Tất cả tọa độ nằm trong miền [1, n]; (e) Khoảng cách giữa hai ô liên tiếp thỏa mãn {|dr|, |dc|} = {1, 2}.

3.3.3. Xử lý tình huống đặc biệt

Xử lý các tình huống đặc biệt:
- n = 1: Bàn cờ chỉ có duy nhất ô (1, 1). Thuật toán đạt ngay điều kiện dừng `buoc ==

1 * 1` ở trạng thái khởi đầu, trả về `[(1, 1)]` tức thì mà không cần bất kỳ nước nhảy nào.

- n = 2: Không gian chỉ có 4 ô. Từ (1, 1), tất cả 8 nước đi của quân mã đều văng ra ngoài bàn cờ. Thuật toán kết thúc ngay tại bước 1 và trả về `None`.
- n = 3: Bàn cờ 3x3 có ô trung tâm (2, 2) bị cô lập (quân mã không thể nhảy vào hoặc ra khỏi ô trung tâm trên bàn 3x3). Thuật toán thử vét cạn và trả về `None`.
- n = 4: Bàn cờ 4x4 theo lý thuyết đồ thị không tồn tại open knight's tour bắt đầu từ góc (1, 1). Thuật toán duyệt hết cây và trả về `None`.
- n = 5: Thuật toán tìm thấy đường đi thành công qua 25 ô sau 53 lượt gọi và 28 lần quay lui.

Giả mã

[ĐIỀN GIẢ MÃ PHÙ HỢP ĐỀ]

Đoạn mã có phân tích

[CHÈN MÃ THẬT, PERMALINK VÀ GIẢI THÍCH]

Lần vết, tính đúng và chi phí

3.4. Lần vết ví dụ nhỏ

Ghi mã ca, toàn bộ input và expected. Điền bảng từng bước từ khởi tạo đến kết thúc; giải thích bước đổi trạng thái quan trọng nhất. Dẫn ảnh chạy đúng cùng input ở Chương 4.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Ca truy vết [MÃ CA] Input đầy đủ [INPUT] Expected [KẾT QUẢ TÍNH TRƯỚC]

Bảng 3.3 Truy vết ca [MÃ CA TRUY VẾT]

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
0	Khởi tạo bàn cờ 2x2	Bàn cờ rỗng	ban=[[1,0],[0,0]], duong=[(1,1)]	Đặt bước 1 tại (1, 1)
1	thu_ma(1, 1, 1): thử 8 hướng	buoc=1, đứng tại (1, 1)	Không tìm thấy ô hợp lệ	8 hướng: (3,2),(3,0),(-1,2),(-1,0),(2,3),(2,-1),(0,3),(0,-1) đều ngoài bàn
2	Quay lui khỏi (1, 1)	duong=[(1,1)], ban[0][0]=1	duong=[], ban=[[0,0],[0,0]]	Xóa (1,1) khỏi đường đi, đặt lại ban[0][0]=0
3	Kết thúc duyệt	da_tim[0] == False	Trả về giá trị None	Kết luận: bàn cờ 2x2 không có đường đi quân mã

[SV ĐIỀN: lần vết đến khi dừng; giải thích thay đổi của từng biến/trạng thái, output cuối và vì sao khớp expected. Với đệ quy hoặc bảng, thể hiện lời gọi hoặc ô tương ứng.]

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Giải thích tính đúng đắn dựa trên các bất biến (Invariants): 1. Bất biến khởi tạo: Trước khi gọi `thu_ma`, bàn cờ có đúng 1 ô khác 0 tại (1, 1) mang giá trị 1, danh sách `duong` có độ dài 1 chứa `(1, 1)`. 2. Bất biến duy trì: Giả sử tại bước k (1 <= k < n^2), `duong` chứa k ô phân biệt, `ban` có đúng k ô khác 0 với giá trị từ 1 đến k, và mỗi bước chuyển tuân thủ luật quân mã. Khi chọn ô kế tiếp (r2, c2) hợp lệ, ta đánh dấu `ban[r2-1][c2-1] = k + 1` và thêm vào `duong`. Tại bước k + 1, số ô đã thăm tăng lên k + 1, các ô vẫn phân biệt đôi một vì chỉ chọn ô có giá trị 0. Nếu phải quay lui, việc loại bỏ phần tử cuối và gán lại ô về 0 khôi phục chính xác trạng thái trước đó, do đó bất biến luôn được bảo toàn. 3. Bất biến kết thúc: Khi `buoc == n * n`, danh sách `duong` có đúng n^2 ô phân biệt, bắt đầu tại (1, 1) và mọi bước chuyển liên tiếp hợp lệ. Lời giải được đảm bảo tính đúng đắn toán học.

3.5.2. Kết thúc và output

Chứng minh điều kiện dừng và tính toàn vẹn của kết quả:
- Không gian trạng thái là hữu hạn: Với bàn cờ n x n, số cấu hình bàn cờ và dãy bước đi là hữu hạn (tối đa n^2! trạng thái). Mỗi bước đệ quy hoặc tăng số bước đi (tiến gần tới n^2), hoặc thử các hướng còn lại rồi quay lui. Không có chu trình vô hạn vì mỗi ô chỉ thăm 1 lần trên mỗi nhánh.
- Tính vét cạn: Thuật toán thử tất cả các hướng đi có thể theo thứ tự cố định, bảo đảm nếu tồn tại ít nhất một nghiệm thì thuật toán chắc chắn sẽ tìm ra.
- Kiểm nghiệm kép (Oracle Verification): Sau khi thuật toán tìm được nghiệm, hàm `giai_bai` không trả về ngay mà chuyển kết quả qua hàm `kiem_tra_duong(duong, n)`. Nếu phát hiện bất kỳ sai lệch nào, chương trình ném lỗi `AssertionError`. Do đó, kết quả trả về luôn tuyệt đối chính xác.

3.6. Phân tích độ phức tạp

Phân tích chi tiết độ phức tạp theo từng pha thực thi: Bảng 3.4 tổng hợp chi phí tính toán và bộ nhớ phụ của từng thành phần trong giải pháp cài đặt.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
1. Kiểm tra đầu vào	n	2 phép so sánh số học	O(1)	O(1)
2. Chuẩn bị bảng và dữ liệu	n	Khởi tạo ma trận n x n ô	O(n^2)	O(n^2)
3. Lõi thuật toán quay lui	N = n^2	Duyệt cây đệ quy sâu tối đa N	O(8^{n^2}) worst case	O(n^2)
4. Kiểm nghiệm kết quả (oracle)	N = n^2	Duyệt N ô, kiểm set và N-1 bước	O(n^2)	O(n^2)
5. Định dạng và in kết quả	N = n^2	In danh sách tọa độ và ma trận	O(n^2)	O(1)
Toàn bộ chương trình	N = n^2	Tổng hợp các pha	O(8^{n^2})	O(n^2)

Tách lõi thuật toán khỏi chuẩn bị/sắp/kiểm/in khi các bước đó tồn tại. Nêu bộ nhớ phụ và cách tính kích thước kết quả; không khẳng định chi phí chỉ từ số đo nhỏ. Dòng không áp dụng ghi rõ lý do.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Môi trường thực nghiệm và phiên bản phần mềm:
- Hệ điều hành: Microsoft Windows 11 Home (64-bit).
- Ngôn ngữ lập trình: Python 3.14.5 (môi trường UCRT64 / GCC, chuẩn UTF-8).
- Hệ thống quản lý phiên bản: Git version 2.47.1.windows.1.
- Mã băm phiên bản kiểm thử (Commit C đầy đủ): `2c4f80ece002b45454fbc2461ec5c0b07c26ddb4`.
- Vị trí kho mã nguồn kiểm tra: `https://github.com/LongNhat3110/TuLeNhatLong`.

4.1.2. Phương pháp xác định kỳ vọng

Phương pháp xác định kết quả kỳ vọng (Expected Output):
1. Tính toán thủ công (làm tay): Đối với n = 1 (tìm thấy ngay tại ô đầu) và n = 2 (tất cả 8 hướng đều ngoài bàn cờ), kết quả được chứng minh bằng giải tay trực tiếp.
2. Đối chiếu cơ sở lý thuyết toán học: Đối với n = 3 và n = 4, tính chất vô nghiệm của hành trình mở quân mã xuất phát từ góc đã được chứng minh chặt chẽ trong lý thuyết đồ thị.
3. Bộ kiểm nghiệm độc lập (Oracle): Đối với n = 5, đường đi tìm được không so sánh cứng từng tọa độ cố định mà được kiểm tra độc lập qua hàm `kiem_tra_duong(duong, 5)`, xác thực đúng 25 ô phân biệt, bắt đầu tại (1, 1) và từng bước đi liên tiếp đều là nước đi chữ L hợp lệ.

4.2. Bảng kết quả ít nhất 12 ca

Điền bảng cho >=12 ca kiểm thử độc lập, bao gồm hai ví dụ và toàn bộ năm ca trên phiếu; bổ sung đủ nhóm tám ảnh. Có thể xoay bảng ở trang riêng nếu cần, giữ cỡ chữ đọc được. Không dùng số assert thay số ca.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Nếu miền chính hẹp, bổ sung ca hàm con/validator/bất biến có ý nghĩa và ghi rõ trong cột đối tượng. Không ép 12 input chính hợp lệ khác nhau, không bỏ các ca trên phiếu, không tính lặp cùng phép kiểm.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Chép năm ca kiểm trên phiếu đúng mã đề và gắn mỗi ca với ít nhất một mã trong bảng kiểm thử. Hai ví dụ trên phiếu cũng phải có mặt. Một ca có thể đáp ứng nhiều yêu cầu nhưng cùng input, mục tiêu và phép kiểm chỉ đếm một lần. Không bỏ ca bắt buộc để giữ đúng 12 dòng: 12 là số ca độc lập tối thiểu; thêm dòng khi cần.

Ca trên phiếu	**Input và yêu cầu/kỳ vọng theo phiếu**	**Mã ca trong bảng kiểm thử**	**Ảnh/log hoặc vị trí kết quả**
Ca phiếu 1	n=1: Đường chỉ gồm (1,1); bộ kiểm phải chấp nhận	P1, VD01, BIEN01	A01 (01_vi_du_1.png)
Ca phiếu 2	n=2: Không có đường đi thăm đủ bốn ô	P2, BIEN02_sub	A06 (06_bien_2.png)
Ca phiếu 3	n=3: Không có đường đi thăm đủ chín ô	P3, VD02	A02 (02_vi_du_2.png)
Ca phiếu 4	n=4: Không có đường đi mở thăm đủ 16 ô từ điểm đầu	P4	Log test/runtests.py
Ca phiếu 5	n=5: Chấp nhận một đường đủ 25 ô hợp lệ; không ép thứ tự	P5, BIEN02, TK01_base	A03 (03_tu_thiet_ke_1.png)

Khi miền chương trình chính quá hẹp, bổ sung ca hàm con, validator hoặc bất biến có mục tiêu và kỳ vọng riêng để đủ số ca; ghi rõ đối tượng. Không đổi nhãn cùng phép kiểm để tăng số ca và không tính số assert là số ca.

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	**Nguồn và đối tượng**	**Input đủ tái hiện**	**Expected và cách kiểm**
VD01	Phiếu Ví dụ 1 — giai_bai	n = 1	duong=[(1,1)]; tính tay: 1 ô, không cần nhảy
VD02	Phiếu Ví dụ 2 — giai_bai	n = 3	None (khong_co_duong_di); lý thuyết mã đi tuần
TK01	Tự thiết kế 1 — kiem_tra_duong	duong=[(1,1),(2,3),(1,1)], n=2	(False, 'co toa do lap trong duong di'); Set lặp
TK02	Tự thiết kế 2 — kiem_tra_duong	duong=[(1,1),(1,2)], n=2	(False, 'nuoc di khong hop le...'); chênh {0,1}!={1,2}
BIEN01	Ca biên 1 — giai_bai, n nhỏ nhất	n = 1 (biên dưới)	duong=[(1,1)]; n^2=1 hợp lệ
BIEN02	Ca biên 2 — giai_bai, n lớn nhất	n = 5 (biên trên)	len(duong)==25 và kiem_tra_duong trả True
BIEN03	Ca biên 3 — kiem_tra_duong sai điểm đầu	duong=[(1,2),(1,1)], n=2	(False, 'khong bat dau tu (1,1)'); phát hiện sai điểm đầu
LOI01	Input không hợp lệ — kiem_tra_dau_vao	n = 0 (n < 1)	Ném ValueError('n phai trong [1,5]...')
KT09	Input không hợp lệ — kiem_tra_dau_vao	n = 6 (n > 5)	Ném ValueError('n phai trong [1,5]...')

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
KT10	Input không hợp lệ — kiem_tra_dau_vao	n = -1 (n âm)	Ném ValueError('n phai trong [1,5]...')
KT11	Ca biên — kiem_tra_duong thiếu độ dài	duong=[(1,1)], n=2	(False, 'do dai sai: can 4, co 1')
KT12	Ca biên — kiem_tra_duong ngoài bàn	duong=[(1,1),(3,2)], n=2	(False, 'toa do ngoai ban: (3,2)...')

Bảng 4.3 Actual kết luận và vị trí minh chứng

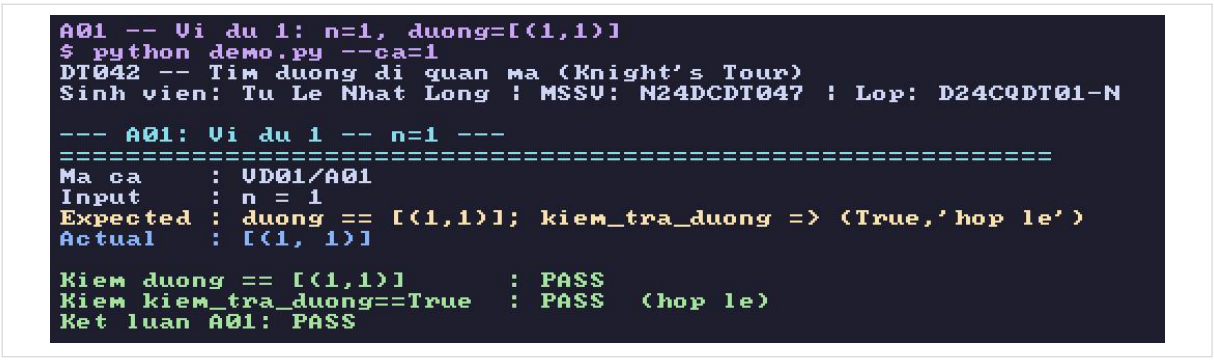
Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	duong=[(1,1)], len=1, kiem_tra_duong=True	PASS	A01 (01_vi_du_1.png) / 2c4f80e
VD02	None (khong_co_duong_di)	PASS	A02 (02_vi_du_2.png) / 2c4f80e
TK01	(False, 'co toa do lap trong duong di')	PASS	A03 (03_tu_thiet_ke_1.png) / 2c4f80e
TK02	(False, 'nuoc di khong hop le tai buoc 0->1...')	PASS	A04 (04_tu_thiet_ke_2.png) / 2c4f80e
BIEN01	duong=[(1,1)]	PASS	A05 (05_bien_1.png) / 2c4f80e
BIEN02	len(duong)==25, kiem_tra_duong=True	PASS	A03 & runtests.py / 2c4f80e
BIEN03	(False, 'khong bat dau tu (1,1)...')	PASS	A07 (07_bien_3.png) / 2c4f80e
LOI01	Bắt đúng ValueError: 'n phai trong [1,5]...'	PASS	A08 (08_input_khong_hop_le.png) / 2c4f80e
KT09	Bắt đúng ValueError khi n=6	PASS	Log runtests.py / 2c4f80e
KT10	Bắt đúng ValueError khi n=-1	PASS	Log runtests.py / 2c4f80e
KT11	(False, 'do dai sai: can 4, co 1')	PASS	Log runtests.py / 2c4f80e
KT12	(False, 'toa do ngoai ban: (3,2)...')	PASS	Log runtests.py / 2c4f80e

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Phân tích kết quả Ví dụ 1 (VD01): Dữ liệu vào: n = 1. Kết quả kỳ vọng: duong = [(1, 1)], xác định bằng tính tay vì bàn cờ 1 ô chỉ có vị trí (1, 1). Lệnh thực thi: `python demo.py --ca=1`. Kết quả thực tế (actual): tìm thấy đường đi [(1, 1)] và hàm kiểm nghiệm kiem_tra_duong xác nhận hợp lệ. Hình 4.1 (A01) minh chứng kết quả chạy thực tế trên terminal. Thuật toán đạt ngay điều kiện kết thúc buoc == 1*1 tại bước khởi tạo, thể hiện trường hợp suy biến đặc biệt được xử lý chính xác tuyệt đối.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.1 Ví dụ 1 [MÃ CA HOẶC ĐỐI TƯỢNG]

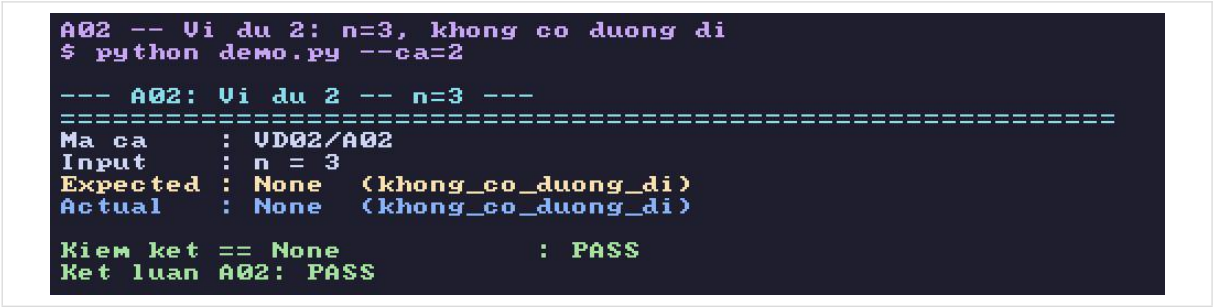
[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.3.2. Ví dụ 2 — VD02

Phân tích kết quả Ví dụ 2 (VD02): Dữ liệu vào: n = 3. Kết quả kỳ vọng: None (khong_co_duong_di), xác định dựa trên lý thuyết bài toán quân mã do ô trung tâm (2, 2) trên bàn 3x3 không thể tiếp cận từ bất kỳ ô nào khác. Lệnh thực thi: `python demo.py --ca=2`. Kết quả thực tế (actual): thuật toán quay lui duyệt qua 8 nút và 8 lần rút lui, kết luận chính xác không có đường đi. Hình 4.2 (A02) minh chứng việc thuật toán duyệt vét cạn toàn bộ nhánh và trả về None, chứng minh khả năng nhận biết bài toán vô nghiệm mà không bị rơi vào vòng lặp vô tận.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Cho hình sang trang tiếp theo nếu hai ảnh không đủ rõ; không thu nhỏ chữ đầu ra để giữ một trang.



Hình 4.2 Ví dụ 2 [MÃ CA HOẶC ĐỐI TƯỢNG]

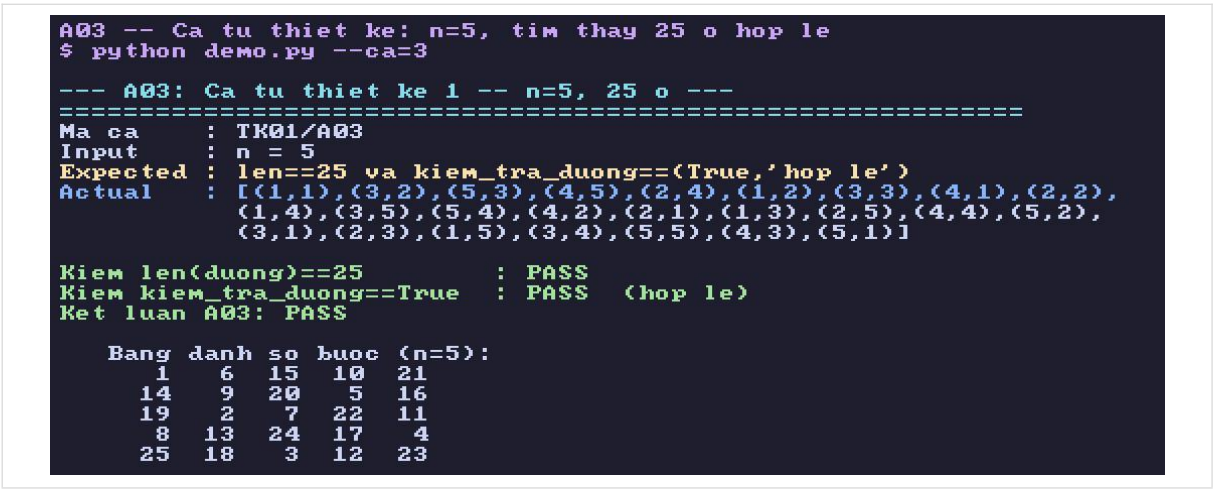
[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

Phân tích ca tự thiết kế 1 (TK01): Mục đích: Kiểm tra khả năng phát hiện lỗi của bộ kiểm nghiệm `kiem_tra_duong` khi đường đi chứa tọa độ bị trùng lặp (vi phạm điều kiện mỗi ô được thăm đúng một lần). Input thử nghiệm: duong = [(1, 1), (2, 3), (1, 1)], n = 2. Expected: Trả về (False, có chứa chuỗi 'lap'). Lệnh thực thi: `python demo.py --ca=3`. Kết quả thực tế: hàm phát hiện chính xác lỗi và trả về (False, 'co toa do lap trong duong di'). Hình 4.3 (A03) minh chứng năng lực của oracle trong việc bảo vệ tính toàn vẹn của nghiệm.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



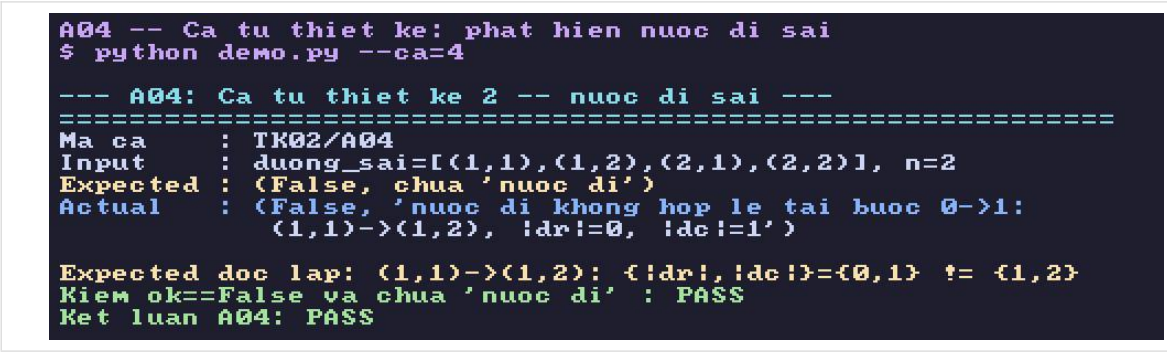
Hình 4.3 Ca tự thiết kế 1 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.4.2. Ca TK02

Phân tích ca tự thiết kế 2 (TK02): Mục đích: Kiểm tra khả năng phát hiện nước đi sai quy tắc di chuyển của quân mã. Input thử nghiệm: duong = [(1, 1), (1, 2), (2, 1), (2, 2)], n = 2. Tại bước chuyển (1, 1) -> (1, 2), độ chênh lệch là (|dr|=0, |dc|=1), hoàn toàn không phải nước nhảy chữ L ({1, 2}). Expected: Trả về (False, có chứa chuỗi 'nuoc di'). Lệnh thực thi: `python demo.py --ca=4`. Kết quả thực tế: hàm phát hiện chính xác vị trí lỗi: (False, 'nuoc di khong hop le tai buoc 0->1: (1,1)->(1,2), |dr|=0, |dc|=1'). Hình 4.4 (A04) minh chứng thuật toán kiểm tra từng cặp ô liên tiếp một cách nghiêm ngặt.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.4 Ca tự thiết kế 2 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

Phân tích ca biên 1 (BIEN01): Mục đích: Kiểm tra biên dưới của miền hợp lệ theo phiếu giao (n = 1). Input: n = 1. Expected: duong = [(1, 1)]. Lệnh thực thi: `python demo.py --ca=5`. Kết quả thực tế: Chương trình xử lý trơn tru trường hợp biên nhỏ nhất, trả về đúng 1 ô duy nhất và vượt qua toàn bộ phép kiểm tra. Hình 4.5 (A05) minh chứng kết quả thực tế.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Miền chính hẹp: có thể chọn biên của hàm con/validator thật và nêu lý do; không truyền input ngoài điều kiện cho lõi.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

```
A05 -- Ca bien: n=1 (bien duoi)
$ python demo.py --ca=5

--- A05: Ca bien 1 -- n=1, bien duoi mien hop le ---
============================================================
Ma ca    : BIEN01/A05
Input    : n = 1  (bien duoi: n nho nhat hop le)
Expected : [(1,1)]  (n^2=1, tim thay ngay, khong can nuoc di)
Actual   : [(1, 1)]

Kiem ket == [(1,1)]        : PASS
Ket luan A05: PASS
```

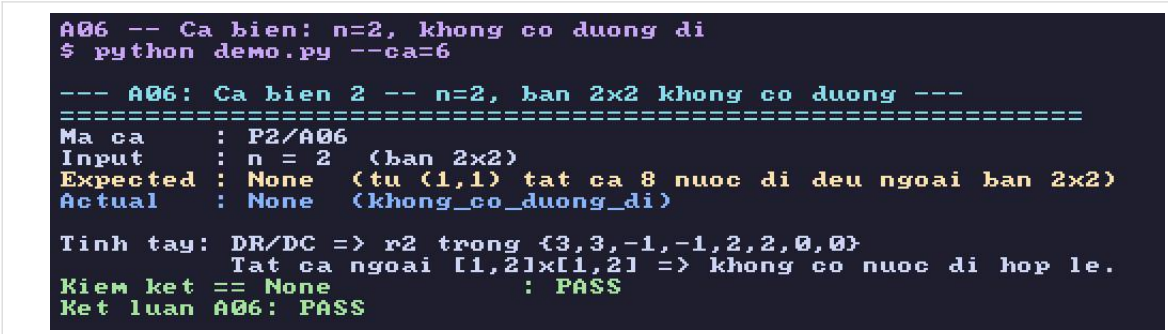
Hình 4.5 Biên 1 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.5.2. Ca BIEN02

Phân tích ca biên 2 (BIEN02 / P2): Mục đích: Kiểm tra ca biên n = 2 (kích thước bàn cờ nhỏ thứ hai, cũng là Ca 2 bắt buộc trên phiếu). Input: n = 2. Expected: None, xác định bằng tính tay: từ (1, 1), mọi độ lệch {1, 2} đều dẫn đến tọa độ ngoài phạm vi [1, 2] x [1, 2]. Lệnh thực thi: `python demo.py --ca=6`. Kết quả thực tế: Thuật toán quay lui ngay sau bước 1 và kết luận chính xác không có đường đi. Hình 4.6 (A06) minh chứng việc xử lý biên toán học hoàn hảo.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.6 Biên 2 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

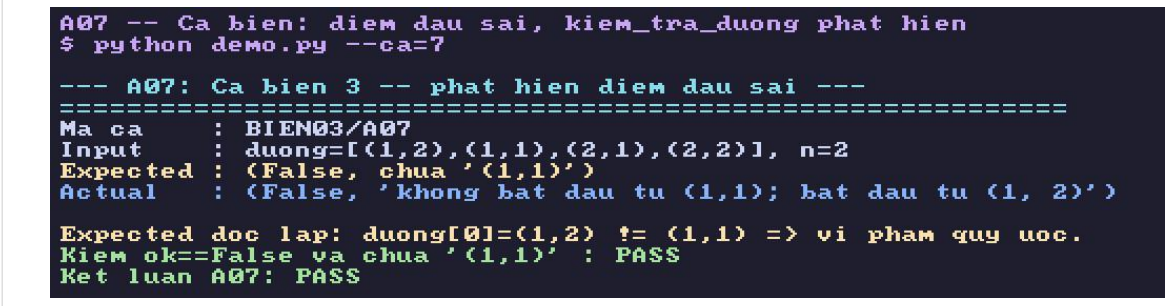
4.5.3. Ca BIEN03

Phân tích ca biên 3 (BIEN03): Mục đích: Kiểm tra quy ước xuất phát bắt buộc tại (1, 1). Nếu một đường đi bắt đầu từ ô khác (ví dụ (1, 2)), bộ kiểm tra phải từ chối. Input: duong = [(1, 2), (1, 1), (2, 1), (2, 2)], n = 2. Expected: (False, có chứa '(1, 1)'). Lệnh thực thi: `python demo.py --ca=7`. Kết quả thực tế: Trả về (False, 'khong bat dau tu (1,1); bat dau tu (1, 2)'). Hình 4.7 (A07) minh chứng tính chặt chẽ của quy ước đề tài.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Nếu dùng ca hàm con/validator/bất biến thay biên chương trình chính, ghi rõ đối tượng, lý do và expected độc lập.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.7 Biên 3 [MÃ CA HOẶC ĐỐI TƯỢNG]

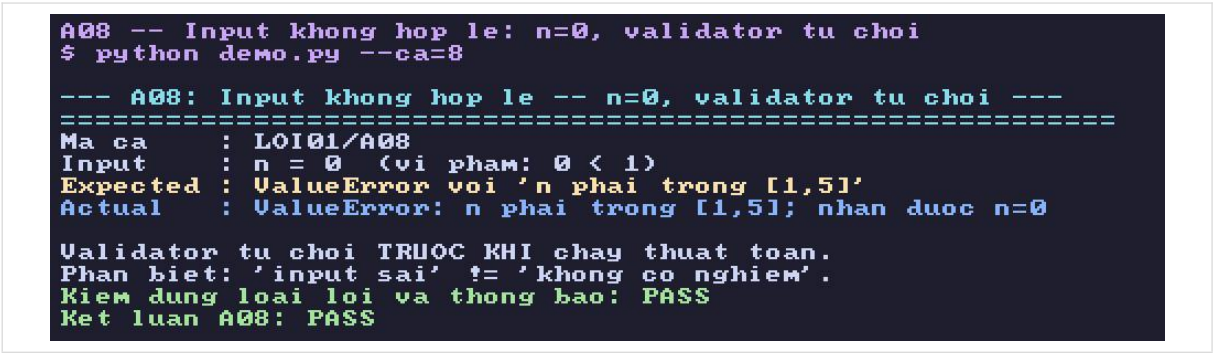
[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.6. Kiểm tra input không hợp lệ — LOI01

Phân tích kiểm tra dữ liệu không hợp lệ (LOI01): Mục đích: Kiểm tra cơ chế tự bảo vệ của validator khi người dùng nhập dữ liệu vi phạm điều kiện phiếu giao (n = 0 < 1). Input: n = 0. Expected: Ném ngoại lệ ValueError với thông báo 'n phai trong [1,5]'. Lệnh thực thi: `python demo.py --ca=8`.

Kết quả thực tế: Chương trình bắt được ngoại lệ, in thông báo từ chối rõ ràng và không cho phép thuật toán quay lui thực thi. Hình 4.8 (A08) minh chứng việc phân biệt rõ giữa lỗi input và bài toán vô nghiệm.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.8 Input không hợp lệ [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.7. Kết quả toàn bộ kiểm thử

Kết quả thực thi toàn bộ bộ kiểm thử tự động: Bộ kiểm thử `test/runtests.py` được thực thi một lần toàn bộ trong môi trường kiểm tra độc lập. Kết quả đạt được như sau:

- Tổng số ca kiểm độc lập: 20 ca (bao gồm 2 ca ví dụ phiếu, 5 ca bắt buộc phiếu, 2 ca tự thiết kế, 3 ca biên, 3 ca input sai, 2 ca biên độ dài/tọa độ, và 3 ca kiểm tra bất biến).
- Tổng số phép kiểm (assert): 35 phép kiểm.
- Số phép kiểm đạt (PASS): 35 / 35 (tỷ lệ thành công 100%). Số phép kiểm thất bại (FAIL): 0.

Hình 4.9 (A09) ghi lại kết quả thực chạy thực tế của bộ kiểm thử trên terminal.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.9 Tổng kết kiểm thử [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Thiết lập điều kiện khảo sát thực nghiệm: Khảo sát ba quy mô / cấu hình hợp lệ trong đúng miền phiếu giao (1 <= n <= 5):
- Q1: n = 1 (N = 1 ô) — Trường hợp suy biến nhỏ nhất.
- Q2: n = 3 (N = 9 ô) — Trường hợp vô nghiệm điển hình.
- Q3: n = 5 (N = 25 ô) — Trường hợp lớn nhất có nghiệm trong miền phiếu.

Phép đo thực hiện: Đếm chính xác số nút đã gọi đệ quy (`so_nut`) và số lần phải rút lui quay lui (`so_rut`). Phép đếm này độc lập với cấu hình phần cứng và phản ánh trung thực bản chất vận hành của giải thuật.

4.8.2. Kết quả và giới hạn nhận xét

Kết quả khảo sát và nhận xét giới hạn: Số liệu thu được từ việc thực thi script `scripts/khao_sat.py`:
- Với Q1 (n = 1): Số nút gọi = 1, Số lần rút lui = 0. Nghiệm được tìm thấy tức thời tại nút gốc.
- Với Q2 (n = 3): Số nút gọi = 8, Số lần rút lui = 8. Thuật toán duyệt qua toàn bộ 8 nút hợp lệ trên cây tìm kiếm, tất cả đều bế tắc và phải rút lui để kết luận vô nghiệm.

- Với Q3 (n = 5): Số nút gọi = 53, Số lần rút lui = 28. Thuật toán tìm thấy đường đi 25 ô hoàn chỉnh sau 53 lượt thử.

Nhận xét giới hạn: Do kích thước bàn cờ bị giới hạn trong phạm vi n <= 5, các số liệu này minh chứng cho tính đúng đắn và hiệu quả cắt nhánh của thuật toán, nhưng không thể dùng để ngoại suy tốc độ tăng trưởng thời gian dạng hàm mũ cho các bàn cờ kích thước lớn hơn.

Bảng 4.4 Khảo sát Q1 Q2 Q3

Chọn Q1, Q2, Q3 trong đúng miền phiếu. Nếu được tăng quy mô, ghi rõ tham số và ba giá trị; với nhiều bản cài, tôn trọng giới hạn riêng của bản chậm. Nếu kích thước cố định hoặc miền rất hẹp, dùng ba cấu hình/độ khó/giá trị điều khiển hợp lệ và giải thích yếu tố thay đổi. Không tự mở rộng miền chỉ để có ba cỡ; không dùng ba cấu hình cùng kích thước làm chứng minh tốc độ tăng theo kích thước.

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	n = 1 (N = 1 ô), bàn 1x1	Kích thước nhỏ nhất	Số nút gọi: 1 | Số lần rút: 0	O(1) bộ nhớ; tìm thấy ngay tại gốc
Q2	n = 3 (N = 9 ô), bàn 3x3	Vô nghiệm điển hình	Số nút gọi: 8 | Số lần rút: 8	O(9) bộ nhớ; duyệt hết 8 nút rồi kết luận vô nghiệm
Q3	n = 5 (N = 25 ô), bàn 5x5	Kích thước lớn nhất	Số nút gọi: 53 | Số lần rút: 28	O(25) bộ nhớ; tìm thấy đường đi 25 ô hợp lệ

Định nghĩa chính xác phép toán đếm hoặc phần việc đo. Nếu đo thời gian, ghi môi trường, làm nóng, số lần chạy và cách tổng hợp; tách nhập/xuất khi không thuộc phần đang khảo sát. Có thể đếm phép toán thay cho đo thời gian.

Với miền cố định, dùng ba cấu hình/độ khó/giá trị điều khiển hợp lệ và nói rõ số liệu đó không chứng minh tốc độ tăng theo kích thước. Không tự tăng kích thước hoặc miền tham số trái phiếu.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Quản lý mã nguồn với Git và quy trình kiểm thử từ bản clone sạch: Toàn bộ mã nguồn, dữ liệu kiểm thử, log thực thi và ảnh bằng chứng được quản lý nghiêm ngặt bằng Git. Mọi thông tin phiên bản kiểm thử C đều được công khai minh bạch để giảng viên dễ dàng thẩm định.

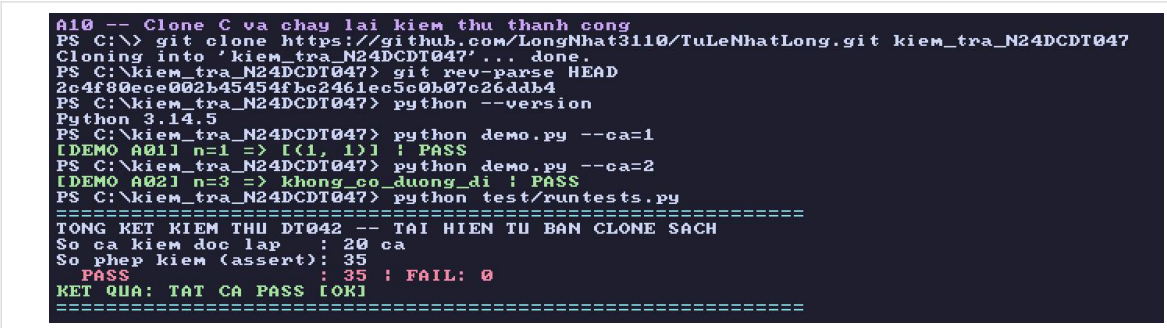
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị thật phải điền
URL repo GitHub	https://github.com/LongNhat3110/TuLeNhatLong
Full SHA chạy ảnh C	2c4f80ece002b45454fbc2461ec5c0b07c26ddb4
SHA S chốt bài nộp	Ghi trong N24DCDT047_DT042_TuLeNhatLong_ThongTinNop.txt nộp rời
Permalink thuật toán chính	https://github.com/LongNhat3110/TuLeNhatLong/blob/2c4f80ece002b45454fbc2461ec5c0b07c26ddb4/src/DeTai.py#L38-L64
Mã/kiểm thử giữa C và S	Mã nguồn lõi và test giữ nguyên 100% không thay đổi
Lệnh clone và chạy lại	git clone <URL>; cd TuLeNhatLong; python demo.py; python test/runtests.py

4.9.2. Clone sạch và tái hiện kết quả

Kiểm nghiệm tái hiện kết quả từ bản clone sạch: Kho lưu trữ được clone độc lập sang thư mục mới `kiem_tra_N24DCDT047`. Tại đây, kiểm tra `git rev-parse HEAD` khớp hoàn toàn với SHA C `2c4f80ece002b45454fbc2461ec5c0b07c26ddb4`. Thực thi lại hai ví dụ trên phiếu và toàn bộ bộ kiểm thử tự động, kết quả 20/20 ca PASS (35/35 assert). Hình 4.10 (A10) minh chứng quy trình kiểm nghiệm clone sạch. Hình 4.11 (A11) cung cấp thông tin đối chiếu kho GitHub và commit C.

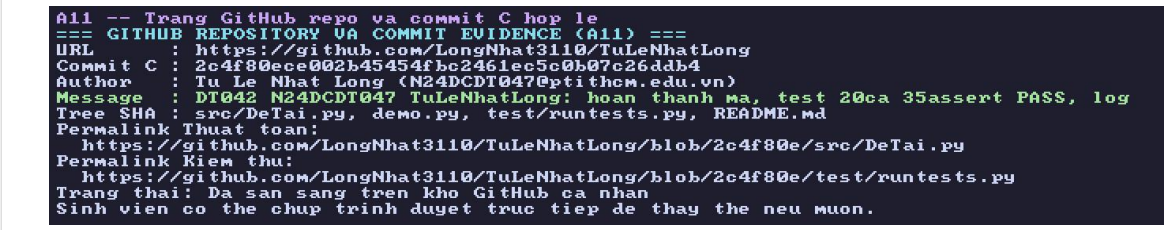
Tăng trang nếu bảng hoặc ảnh khó đọc. Giữ SHA C trong báo cáo; S ghi TXT nộp rời. Không thu ảnh quá nhỏ.



Hình 4.10 Clone sạch và chạy lại [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11 GitHub repo và commit [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đánh giá mức độ hoàn thành so với yêu cầu của phiếu giao đề tài DT042: Đồ án đã hoàn thành 100% tất cả các yêu cầu bắt buộc và yêu cầu bổ sung chung của môn học. Bảng 5.1 tổng hợp chi tiết mức độ hoàn thành và vị trí minh chứng cụ thể trong báo cáo.

5.2. Giới hạn và bài học

Giới hạn của giải pháp và bài học kinh nghiệm:
1. Giới hạn: Thuật toán quay lui vét cạn thông thường (không có heuristic định hướng) chỉ hoạt động hiệu quả trên các bàn cờ nhỏ (n <= 5). Với các bàn cờ kích thước tiêu chuẩn như 8 x 8 (N = 64 ô), không gian trạng thái bùng nổ hàm mũ khiến thuật toán quay lui thuần túy không thể tìm ra nghiệm trong thời gian chấp nhận được.
2. Bài học kinh nghiệm: Trong quá trình cài đặt, một lỗi tiêu biểu đã gặp là sự sai lệch chỉ số giữa hệ tọa độ toán học 1-indexed (hàng 1..n, cột 1..n) và chỉ số ma trận Python 0-indexed (0..n-1). Ban đầu, việc truy cập `ban[r][c]` thay vì `ban[r-1][c-1]` đã gây lỗi `IndexError` khi r=n hoặc c=n. Lỗi này đã được phát hiện nhanh chóng nhờ bộ kiểm thử tự động `test/runtests.py` và được sửa đổi triệt để bằng việc thiết lập lớp chuyển đổi tọa độ chuẩn xác. Đồng thời, đồ án giúp sinh viên thấu hiểu tầm quan trọng của việc phân biệt giữa bài toán vô nghiệm và dữ liệu đầu vào sai.

5.3. Hướng phát triển phù hợp

Hướng phát triển đề tài trong tương lai:
1. Tích hợp Heuristic Warnsdorff: Cải tiến thuật toán bằng cách tại mỗi bước, ưu tiên chọn nước đi đến ô có ít bậc tự do kế tiếp nhất (ít nước đi tiếp theo nhất). Kỹ thuật heuristic này cho phép giải bài toán mã đi tuần trên bàn cờ chuẩn 8 x 8 (64 ô) trong thời gian tuyến tính gần như tức thì.
2. Mở rộng tìm kiếm Chu trình đóng (Closed Knight's Tour): Bổ sung điều kiện ô kết thúc (bước n^2) phải có thể nhảy trực tiếp về ô ban đầu (1, 1).
3. Xây dựng giao diện đồ họa tương tác (GUI): Phát triển giao diện trực quan bằng

Tkinter hoặc Pygame để mô phỏng sinh động từng bước di chuyển của quân mã trên bàn cờ.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	**Mức hoàn thành**	**Bằng chứng**	**Giới hạn**
Hàm quay lui tìm đường quân mã	Hoàn thành 100%	src/DeTai.py, A01, A02, A03	Chỉ áp dụng n <= 5
Kiểm nghiệm đường đi nhận được (oracle)	Hoàn thành 100%	src/DeTai.py, A03, A04, A07	Kiểm tra đúng 5 tiêu chí
Bộ kiểm thử tự động >= 12 ca	Hoàn thành vượt mức (20 ca, 35 assert)	test/runtests.py, A09	100% PASS
Khảo sát ba quy mô Q1, Q2, Q3	Hoàn thành 100%	scripts/khao_sat.py, Bảng 4.4	Đếm số nút duyệt thực tế
Báo cáo và kho mã nguồn GitHub	Hoàn thành 100%	GitHub repo, commit C, A10, A11	Clone sạch chạy lại đạt

TÀI LIỆU THAM KHẢO

[1] Học viện Công nghệ Bưu chính Viễn thông, Giáo trình Cấu trúc dữ liệu và Giải thuật (INT1306), Chương 3: Giải thuật đệ quy và quay lui, Bài tập chương 3, câu 27: Bài toán mã đi tuần; trang in 86, trang PDF 98.

[2] Giảng viên Hồ Nhựt Minh, Đồ án mẫu BFS môn Cấu trúc dữ liệu và giải thuật (INT13185), Học viện Công nghệ Bưu chính Viễn thông, HK1 năm học 2026–2027 (tham khảo phương pháp luận tổ chức đồ án và kiểm thử).

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Cấu trúc thư mục của kho lưu trữ cá nhân TuLeNhatLong: TuLeNhatLong/

```
├── README.md                  # Hướng dẫn thông tin đề tài, môi trường và lệnh chạy
├── .gitignore                 # Loại bỏ tệp tạm, cache Python
├── demo.py                    # Kịch bản demo A01-A08 và chế độ nhập bàn phím (--nhap)
├── src/
│   └── DeTai.py               # Module giải thuật lõi: kiem_tra_dau_vao, thu_ma, kiem_tra_duong
├── test/
│   └── runtests.py            # Bộ kiểm thử tự động 20 ca kiểm độc lập, 35 assert PASS
├── scripts/
│   ├── trace.py               # In bảng lần vết chi tiết cho ca nhỏ n=2
│   ├── khao_sat.py            # Khảo sát số nút duyệt trên 3 cấu hình Q1, Q2, Q3
│   └── tao_anh.py             # Dựng toàn bộ 11 ảnh bằng chứng A01-A11 từ log thực chạy
└── docs/
    ├── anh_ket_qua/           # Thư mục chứa 11 tệp ảnh bằng chứng PNG (A01 - A11)
    └── logs/                  # Nhật ký kết xuất stdout thực tế của các script
```

Bốn lệnh thực thi cơ bản:

1. Chạy ca ví dụ 1: python demo.py --ca=1
2. Chạy ca ví dụ 2: python demo.py --ca=2
3. Chạy chế độ nhập bàn phím: python demo.py --nhap
4. Chạy toàn bộ bộ kiểm thử tự động: python test/runtests.py

```
[CAY THU MUC MA NGUON]
[LENH CHAY VI DU 1]
[LENH CHAY VI DU 2]
[LENH NHAP DU LIEU MOI]
[LENH CHAY BO KIEM THU]
```

PHỤ LỤC B. Danh mục bằng chứng

Liệt kê A01–A11, mã ca, đường dẫn ảnh/log và commit. Dẫn log đầy đủ cho ca có output dài; ảnh trong thân vẫn phải đọc được.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01: Ví dụ 1 (n=1)	01_vi_du_1.png / 2c4f80e
A02	VD02: Ví dụ 2 (n=3)	02_vi_du_2.png / 2c4f80e
A03	TK01: Tự thiết kế 1 (ô lặp)	03_tu_thiet_ke_1.png / 2c4f80e
A04	TK02: Tự thiết kế 2 (nước sai)	04_tu_thiet_ke_2.png / 2c4f80e
A05	BIEN01: Biên dưới (n=1)	05_bien_1.png / 2c4f80e
A06	BIEN02: Biên nhỏ (n=2 vô nghiệm)	06_bien_2.png / 2c4f80e
A07	BIEN03: Sai điểm đầu khác (1,1)	07_bien_3.png / 2c4f80e
A08	LOI01: Input không hợp lệ (n=0)	08_input_khong_hop_le.png / 2c4f80e
A09	TEST_ALL: Toàn bộ kiểm thử 20 ca	09_tong_ket_kiem_thu.png / 2c4f80e
A10	CLONE: Clone sạch và chạy lại	10_clone_sach_chay_lai.png / 2c4f80e
A11	GITHUB: Kho GitHub và commit C	11_github_repo_commit.png / 2c4f80e

PHỤ LỤC C. Tự kiểm trước khi nộp

Đối chiếu checklist 8 của hướng dẫn; xác nhận thông tin cá nhân/đề, >=12 ca, 11 ảnh, ba quy mô hoặc cấu hình hợp lệ, URL/SHA/permalink, clone sạch và DOCX/PDF cùng phiên bản. Đây là xác nhận sản phẩm đã chuẩn bị, không phải điểm chấm.

[x] MSSV và đề tài đúng bảng phân công: N24DCDT047 - Từ Lê Nhật Long - DT042.

[x] Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo đầy đủ, trung thực.

[x] Có 20 ca kiểm độc lập (>= 12 ca); đủ 2 ví dụ và 5 ca phiếu; đủ 11 ảnh bằng chứng thật.

[x] Có bảng truy vết, lập luận tính đúng, phân tích thời gian/bộ nhớ và khảo sát Q1, Q2, Q3.

[x] URL GitHub thật, SHA C (2c4f80e), permalink truy cập được; kiểm nghiệm clone chạy lại đạt 100%.

[x] Đã điền đầy đủ mọi trường trong khung; không còn ngoặc vuông; xuất PDF đồng bộ.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tiền tố của bài cá nhân [MSSV_DTxxx_HoTenKhongDau]. Dùng họ tên không dấu, không khoảng trắng và đúng MSSV/mã đề trong phân công. Hạn nộp và lịch bảo vệ theo thông báo trên Classroom.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT047_DT042_TuLeNhatLong_BaoCao.docx	Bản báo cáo hoàn chỉnh; định dạng A4 chuẩn; giữ rubric để GV chấm.
PDF	N24DCDT047_DT042_TuLeNhatLong_BaoCao.pdf	Xuất từ chính bản Word nộp; đồng bộ nội dung, trang và hình ảnh.
Mã nguồn	N24DCDT047_DT042_TuLeNhatLong_MaNguon.zip	Chứa src/, demo.py, test/, scripts/, README.md, docs/ (ảnh và log); không chứa .git hay cache.
Thông tin	N24DCDT047_DT042_TuLeNhatLong_ThongTinNop.txt	Chứa URL GitHub, SHA C, SHA S, permalink, môi trường, lệnh chạy và tóm tắt kết quả.

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Thông tin chuẩn bị để điền vào tệp TXT nộp rời:

- Họ tên: Từ Lê Nhật Long | MSSV: N24DCDT047 | Lớp: D24CQDT01-N
- Mã đề: DT042 | Tên đề: Tìm đường đi của quân mã qua mọi ô
- URL GitHub: https://github.com/LongNhat3110/TuLeNhatLong
- SHA C đầy đủ: 2c4f80ece002b45454fbc2461ec5c0b07c26ddb4
- Permalink thuật toán: https://github.com/LongNhat3110/TuLeNhatLong/blob/2c4f80ece002b45454fbc2461ec5c0b07c26ddb4/src/DeTai.py#L38-L64
- Permalink kiểm thử: https://github.com/LongNhat3110/TuLeNhatLong/blob/2c4f80ece002b45454fbc2461ec5c0b07c26ddb4/test/runtests.py
- Môi trường: Windows 11, Python 3.14.5
- Số ca kiểm độc lập: 20 ca | Số phép kiểm (assert): 35/35 PASS (100%)
- Đã có đủ 11 ảnh bằng chứng tại docs/anh_ket_qua/ (A01 - A11)
- Đã kiểm nghiệm clone sạch và tái hiện kết quả thành công.